

Algoritmos y Estructuras de Datos

Examen extraordinario - Completo

Instrucciones

Lea atentamente las instrucciones siguientes antes de continuar:

- Para este examen **solo se permite el uso de material de escritura**, como bolígrafos, lápices, gomas o cinta correctora. No está permitido usar dispositivos electrónicos de ningún tipo, apuntes ni ningún otro material distinto del indicado. **No se corregirá ninguna parte del examen escrita en lápiz.**
- Si tiene más de una hoja, este examen se entrega grapado y **no deberá desgraparlo.**
- Los **enunciados** de los problemas incluyen **espacio para responder** en forma de cajas. **Solo se corregirán estos espacios.** Antes de continuar leyendo, por favor, escriba su nombre completo y DNI en todas las páginas que tengan cabecera.
- Se le proporcionará una hoja de sucio adicional.
- Al finalizar el examen deberá **tomar una fotografía de todos las páginas que tengan cabecera.**
- La correcta interpretación del enunciado se considera parte de las competencias a evaluar en el examen.
- Durante el examen aparecerán fragmentos de código, si en alguna parte de ellos aparece el texto "...", eso indicará que dicho fragmento en concreto no se proporciona porque no es relevante para la resolución del ejercicio.
- Cada ejercicio incluye todas las clases y métodos necesarios para resolverlo que no pertenezcan a las librerías estándar de Java.

Ya puede comenzar el examen. ¡Buena suerte!

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Ejercicio 1: 2 puntos

Se tiene una lista enlazada simple de floats, en la que cada nodo (NodoLista) contiene una referencia al siguiente, pero no al anterior. Implementar el método **recursivo** `invertirRecursivo()`, cuya cabecera se le proporciona a continuación:

```
public class NodoLista { int dato; NodoLista siguiente; }

public static NodoLista invertirRecursivo(NodoLista actual) { // TODO: A completar }
```

Este método deberá devolver el primer nodo de otra lista del mismo tipo, con los mismos nodos, pero en orden inverso. Es decir, que si recibe la el nodo A de la siguiente lista: $A \rightarrow B \rightarrow C \rightarrow D$

Deberá devolver el nodo D de la siguiente lista: $D \rightarrow C \rightarrow B \rightarrow A$

El caso base se dará cuando el nodo proporcionado (`actual`) es `null` o el nodo al que apunta el nodo proporcionado (`actual.siguiente`) es `null`.

Escriba su respuesta a continuación:

```
public static NodoLista invertirRecursivo(NodoLista actual) {
    // Caso base: lista vacía o último elemento
    if (actual == null || actual.siguiente == null) {
        return actual;
    }

    // Paso recursivo: invertir el resto de la lista
    // nuevaCabecera será el último elemento de la lista original
    NodoLista nuevaCabecera = invertirRecursivo(actual.siguiente);

    // Lógica de inversión:
    // actual.siguiente es el nodo que va después del actual.
    // Hacemos que ese nodo apunte hacia atrás (a 'actual').
    actual.siguiente.siguiente = actual;

    // El actual pasa a ser el último (por ahora), así que apunta a null
    actual.siguiente = null;

    return nuevaCabecera;
}
```

- Caso base: 0,5 puntos
- Invocación recursiva: 0,5 puntos
- Lógica de inversión: 0,5 puntos
- Return recursivo: 0,5 puntos

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Ejercicio 2: 1,5 puntos

Examine el siguiente método recursivo que opera sobre un array **ordenado**:

```
public static int busqueda(int[] arr, int inicio, int fin, int x) {  
    if (fin < inicio) return -1;  
    int medio = inicio + (fin - inicio) / 2;  
    if (arr[medio] == x) return medio;  
    if (arr[medio] > x)  
        return busqueda(arr, inicio, medio - 1, x);  
    return busqueda(arr, medio + 1, fin, x);  
}
```

- Dado $arr = \{2, 5, 8, 12, 16, 23, 38, 56, 72, 91\}$ y buscando $x = 23$. Indique la secuencia de valores que toma la variable `medio` hasta encontrar el valor. (0,5 puntos)
- ¿Qué ocurre si el array no está ordenado previamente? (0,5 puntos)
- ¿Cuál es la complejidad del algoritmo en notación O ? Justifique basándose en cómo se reduce el tamaño del problema en cada llamada recursiva. (0,5 puntos)

Escriba su respuesta a continuación:

- 1. $inicio=0, fin=9, medio = 4$ (Valor 16). $16 < 23$, buscar en derecha.
- 2. $inicio=5, fin=9, medio = 7$ (Valor 56). $56 > 23$, buscar en izquierda.
- 3. $inicio=5, fin=6, medio = 5$ (Valor 23). ¡Encontrado!
- Si no está ordenado el algoritmo falla. La búsqueda binaria asume que si x es mayor que el `medio`, debe estar a la derecha. Sin orden, esta premisa es falsa y no encontrará el elemento correctamente.
- Complejidad $O(\log n)$. En cada llamada recursiva, el espacio de búsqueda ($fin - inicio$) se divide por 2. El número de pasos es logarítmico respecto al tamaño del array.

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Ejercicio 3: 3,5 puntos

En un videojuego multijugador, varios hilos intentan actualizar una variable compartida que representa la salud de un jefe final. Un programador novato ha escrito el siguiente código:

```
public class JefeFinal {  
    private int salud = 10000;  
  
    public void recibirDano(int cantidad) {  
        int saludActual = salud;  
        salud = saludActual - cantidad;  
    }  
}
```

1. Explique qué es una condición de carrera y describa paso a paso cómo se manifiesta en este código si dos *threads* llaman simultáneamente a `recibirDano(100)`. ¿Cuál sería la salud final en ese escenario? ¿Cuál debería ser? (1 punto)
2. Haga los cambios que considere necesarios en la clase `JefeFinal` para solucionar el problema. Asegúrese de proteger adecuadamente la sección crítica. (1 punto)
3. Escriba un método `autoResurreccion()` que espere a que la salud del `JefeFinal` sea menor o igual que 0 y, cuando eso ocurra, la restaure a 10000. (1 punto)
4. Modifique el método `recibirDano(int cantidad)` para que reanude los *threads* que estén esperando en `autoResurreccion()`. 0,5 puntos

Escriba su respuesta a continuación:

Una condición de carrera ocurre cuando dos o más *threads* acceden a un recurso compartido concurrentemente y el resultado final de la ejecución depende del orden de planificación de dichos hilos por parte del sistema operativo. Con dos *threads* (A y B) podría suceder lo siguiente:

1. A entra en `recibirDano(100)`, lee `salud` y almacena este valor en su variable local (`saludActual = 10000`).
2. En este momento el planificador da paso a B. B ejecuta `recibirDano(100)`, lee la variable global (que sigue siendo 10000) y guarda el valor en su variable local (`saludActual = 10000`).
3. El planificador da paso a A, que ejecuta la asignación: `salud = 10000 - 100` (guarda 9900 en `salud`).
4. El planificador da paso a B, que ejecuta la asignación: `salud = 10000 - 100` (guarda 9900 en `salud`).

La salud final del jefe se queda en 9900, y debería haber sido 9800.

```
// Al añadir synchronized el método se convierte en una sección crítica  
public synchronized void recibirDano(int cantidad) {  
    int saludActual = salud;  
    salud = saludActual - cantidad;  
    // Al añadir notifyAll() se avisa a los threads que estén esperando  
    notifyAll()  
}  
  
public synchronized void autoResurreccion() {  
    while (salud >= 0)  
        wait();  
    salud = 10000;  
}
```

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Ejercicio 4: 1,5 puntos

Se desea crear un sitio web de Bingo online. Cuando un usuario realiza una petición GET el servicio le muestra un mensaje de bienvenida y un formulario con un botón (imagen izquierda). Cuando se pulsa el botón, el servidor generará un número aleatorio entre el 1 y el 90, y lo devolverá en una tabla de una única columna, junto con el formulario original. La imagen derecha muestra el resultado tras haber pulsado el botón 3 veces.

Bienvenidos al Bingo.

Bienvenidos al Bingo.

Pulsa el botón para generar un nuevo número. Pulsa el botón para generar un nuevo número.

Nuevo número

Nuevo número

87

56

59

(0,5 puntos) Escriba a continuación el contenido del fichero bingo.html que se usará como plantilla en el servlet a desarrollar en el siguiente apartado. No hace falta que incluya estilos en este momento.

```
<!DOCTYPE html>
<html>
<head></head>

<body>
<h1>Bienvenidos al Bingo.</h1>
<h2>Pulsa el botón para generar un nuevo número.</h2>

<form action='bingo' method='POST'>
  <input type='submit' name='enviar' value='Nuevo número'>
</form>
<h2></h2>
<table><tr></tr></table>
</body>
</html>
```

- Form correcto (action, method, etiquetas) +20 %
- Input correcto (input, submit, value) +20 %
- Mensajes y huecos correctos +20 %
- Todo el HTML completo y correcto +40 %

(1 punto) Escriba a continuación el código de un servlet que debe tratar las peticiones enviadas:

- Ante una petición HTTP GET, el servicio prepara la página bingo.html y la envía, eliminando cualquier número que hubiera salido previamente. Es decir, hace un reinicio.
- Ante una petición HTTP POST, el servicio generará un nuevo número aleatorio, y lo incluirá en la parte inferior de la tabla de números, a continuación de los anteriores. En la figura derecha, primero se generó el 87, luego el 56 y por último el 59. Los números mostrados no pueden volver a aparecer (no se pueden repetir) hasta que no haya un reinicio. Además, si el número obtenido fuese el 22 se mostrará un mensaje en verde que diga "¡LOS DOS PATITOS!".

Suponga que tiene disponible en su servlet el método `String getPlantilla (String fichero)` que, dado el nombre de un fichero disponible en su proyecto, devuelve el contenido de ese fichero como un `String`. Por ejemplo, `getPlantilla("alta.html")` devuelve el contenido del fichero `alta.html` de su proyecto como un `String`. Además, el método `int nextInt(int m)` de la clase `Random` devuelve un `int` entre 0 y `m`, ambos incluidos.

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

```
// Fijar la ruta indicada en el enunciado y que coincida con la indicada
// en el action del form +20%
@WebServlet("/bingo")
public class Bingo extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Resetear los números anteriores que pudiera haber +10%
        request.getSession().removeAttribute("numeros");

        // Configurar la plantilla y enviar mensaje correcto +10%
        String html = getPlantilla("buscar.html");
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print(html);
        out.flush();
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String pagina = getPlantilla("buscar.html");

        // Recuperar la lista de números o crearla si no existe +10%
        List<Integer> numeros = (List)request.getSession().getAttribute("numeros");
        if(numeros==null)
            numeros = new ArrayList<Integer>();

        //Generar un número que no haya salido +10%
        Random r = new Random();
        int numero = r.nextInt(89)+1;
        for(; numeros.contains(numero); numero=r.nextInt(89)+1) ;

        //Añadir el número al listado y guardarlo +10%
        numeros.add(numero);
        request.getSession().setAttribute("numeros", numeros);

        //Generar la tabla resultante +10%
        String tabla = "";
        for(Integer n : numeros)
            tabla += "<tr><td>"+n+"</td></tr>";
        pagina = pagina.replace("<tr></tr>", tabla);

        //Controlar el 22 aplicando el estilo +20%
        if(numero==22)
            pagina = pagina.replace("<h2></h2>", "<h2 style='color:green;'>2 patitos!</h2>");

        // Reemplazar en la plantilla y enviar (no puntúa, ya que se puntúa en el doGet)
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print(html);
        out.flush();
    }
}
```

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Ejercicio 5: 1,5 puntos

(0,50 puntos) Deseamos generar un servicio que espera conexiones TCP en el puerto 2222, y que ante la recepción de un mensaje que contenga un número entero entre 1 y el 90, devuelva la cadena de texto que representa al número en el Bingo, y luego desconecta al cliente para atender a nuevas conexiones. No hace falta que gestione errores o mensajes incorrectos. Puede suponer que dispone de un atributo estático `Map<Integer, String>mapa` en su clase que ofrece la correspondencia entre los números y las cadenas de texto que los representan. A continuación puede ver ejemplos de estas correspondencias.

2,El patito
8,Las gafas
22,Los dos patitos

```
private static Map<Integer, String> mapa;

public static void main(String[] args) throws Exception {
    // Levantar servidor y esperar conexión +20%
    ServerSocket s = new ServerSocket(2222);
    while(true) {
        Socket r = s.accept();
        // Obtener los streams +20%
        BufferedReader in = new BufferedReader(new InputStreamReader(r.getInputStream()));
        PrintWriter out = new PrintWriter(r.getOutputStream());
        // Leer la línea que me envían y sacar el entero +20%
        String str = in.readLine();
        int i = Integer.parseInt(str);
        // Recuperar y devolver el texto +20%
        out.println(mapa.get(i));
        out.flush();
        // Cerrar el cliente para esperar a otros +20%
        r.close();
    }
}
```

(1 punto) Ofrezca una respuesta corta a las siguientes cuestiones.

Si un navegador web recibe un archivo que contiene la estructura de una página y otro que define que los botones sean azules, ¿qué tecnologías (lenguajes) corresponden a cada archivo?

HTML para el primero y CSS para el segundo.

¿Dónde se ejecuta el código JavaScript de una página web convencional (como `alert("Hola")`): en el ordenador del usuario o en el ordenador de la empresa dueña de la web?

En el ordenador del usuario (el cliente). El navegador descarga el script y lo procesa localmente.

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 29/06/26	

Cuando escribes `www.google.com` en el navegador y pulsas Intro, ¿qué tipo de mensaje HTTP genera tu navegador hacia el servidor?

Petición HTTP GET

Si un servidor web te devuelve un código de estado HTTP 404, ¿qué significa y de quién es la culpa normalmente?

Recurso no encontrado. El cliente ha pedido algo que no existe.

Imagina que programas un servlet para calcular el precio final de un carrito con impuestos. ¿El navegador del usuario recibe el código Java de ese servlet?

No, el código se ejecuta en el servidor y el navegador recibe la respuesta que genere.

Algunos métodos disponibles en HttpServletRequest, junto con el tipo de valor devuelto	
Cookie []	getCookies() Devuelve un array con todas las cookies que el cliente ha enviado en esta petición.
String	getHeader(String name) Devuelve el valor de la cabecera indicada.
Enumeration<String>	getHeaders(String name) Devuelve todos los valores de la cabecera indicada.
String	getParameter(String name) Devuelve el valor del parámetro indicado, o null si no se ha recibido en la petición.
Enumeration<String>	getParameterNames() Devuelve los nombres de todos los parámetros recibidos en esta petición.
ServletContext	getServletContext() Devuelve un objeto que representa el contexto de ejecución.
HttpSession	getSession() Devuelve la sesión asociada con esta petición. Si no existe, crea una y la devuelve.
Algunos métodos disponibles en HttpServletResponse, junto con el tipo de valor devuelto.	
void	addCookie(Cookie c) Añade la cookie c a la respuesta.
void	addHeader(String name, String value) Añade la cabecera name a la respuesta con valor value.
OutputStream	getOutputStream() Devuelve un OutputStream para enviar bytes en la respuesta.
PrintWriter	getWriter() Devuelve un PrintWriter para escribir texto en la respuesta.
void	setContentType(String type) Fija el tipo de contenido de la respuesta.
Algunos métodos disponibles en HttpSession, junto con el tipo de valor devuelto	
Object	getAttribute(String name) Devuelve el objeto asociado a name en esta sesión, o null si no hay ningún objeto asociado a ese nombre.
Enumeration<String>	getAttributeNames() Devuelve todos los nombres asociados a los objetos almacenados en esta sesión.
void	removeAttribute(String name) Elimina el objeto asociado a name en esta sesión.
void	setAttribute(String name, Object value) Almacena el objeto value asociado al nombre name en esta sesión.
Algunos métodos disponibles en Cookie, junto con el tipo de valor devuelto.	
	Cookie(String name, String value) Crea una cookie con el valor especificado para name y value.
String	getName() Devuelve el nombre de esta cookie.
String	getValue() Devuelve el valor de esta cookie.
Algunos métodos disponibles en ServletContext, junto con el tipo de valor devuelto.	
Object	getAttribute(String name) Devuelve el objeto asociado a name en el contexto de ejecución del Servlet, o null si no está definido.
Enumeration<String>	getAttributeNames() Devuelve los nombres asociados a los objetos almacenados en este contexto.
void	removeAttribute(String name) Elimina el objeto asociado a name en este contexto.
void	setAttribute(String name, Object value) Almacena el objeto value asociado al nombre name en este contexto.